CS 106L — Lecture 2

Types, Structs, `auto` & Structured Bindings · Edited study notes

Learning objectives

- Explain static typing and function overloading.
- Use a struct to group related data and pass it between functions.
- Understand `std::pair` and `std::tuple` at a basic level.
- Use `auto` and structured bindings to make code easier to read.

1. Types and static typing

A type specifies what kind of value a variable holds and which operations are meaningful for it. In C++, a variable's type is known at compile time.

```
int age = 20;
double temperature = 77.3;
std::string name = "Ada";
bool finished = true;
std::size_t count = 5;
```

- `int`: integer values.
- `double`: floating-point values.
- `std::string`: text.
- `bool`: `true` or `false`.
- `std::size_t`: an unsigned integer type commonly used for sizes and counts.

Why static typing?

- It enables the compiler to catch many type-related errors before the program runs.
- It makes declarations and interfaces more explicit.
- It helps compilers generate efficient code.

Integer division

```
int a = 5 / 2;          // a is 2
double b = 5.0 / 2;    // b is 2.5
double c = 5 / 2;      // c is 2.0
```

The expression `5 / 2` performs integer division because both operands are integers. Assigning its result to a `double` does not restore the discarded fractional part.

Function return types

```
int half(int x) {
    return x / 2;
}

double half_as_double(double x) {
    return x / 2;
}

void print_value(double x) {
    std::cout << x << std::endl;
}
```

A function's return type describes the type of value it returns. `void` means the function does not return a value.

2. Function overloading

C++ allows multiple functions to share a name when their parameter lists differ in a way that distinguishes the overloads. The compiler selects the appropriate overload based on the arguments.

```
double scale(int x) {
    return static_cast<double>(x) + 3;
}

double scale(double x) {
    return x * 3;
}

// scale(2)   calls the int overload -> 5.0
// scale(2.0) calls the double overload -> 6.0
```

> The return type alone is not enough to distinguish overloaded functions; the parameter list matters.

3. Structs: group related fields

A `struct` groups named variables—called fields—under one type. Each field has its own type.

```
struct Student {
    std::string name;
    std::string state;
    int age;
};

Student s = {"Ethan", "CA", 20};
std::cout << s.name << std::endl;
```

- Use the dot operator (`.`) to access a field, as in `s.name`.
- The order in a brace initializer corresponds to the order in which the fields are declared.
- A struct lets a function accept or return a single value containing several related pieces of information.

Passing and returning a struct

```
void printStudent(const Student& student) {
    std::cout << student.name << " from "
              << student.state << " ("
              << student.age << ")" << std::endl;
}

Student makeStudent() {
    return {"Ethan", "CA", 20};
}
```

This example uses a `const` reference in the parameter to avoid copying the struct and to indicate that the function will not modify it. The core lecture idea is that the struct packages related fields into one value.

4. Pairs and tuples

`std::pair` groups two values, which may have different types. The fields are accessed using `first` and `second`.

```
std::pair<bool, Student> result;
result.first = true;
result.second = {"Ethan", "CA", 20};
```

A pair can represent a status together with a result. The lecture also demonstrates nested pairs for returning a success flag and two quadratic roots; however, nested pairs can become difficult to read.

```
std::tuple<std::string, int, int> record;
```

```
std::string name = std::get<0>(record);
int first_number = std::get<1>(record);
```

`std::tuple` groups multiple values and uses index-based access such as `std::get<0>`. In ordinary application code, a named struct is often clearer when the fields have meaningful names.

5. Type deduction with `auto`

`auto` asks the compiler to deduce a variable's type from its initializer. It does not mean that the variable is untyped.

```
auto a = 3;                     // int
auto b = 4.3;                   // double
auto c = 'X';                   // char
auto p = std::make_pair(3, 3);  // pair<int, int>
```

- Use `auto` when the type is obvious from the initializer or when writing the full type would make code harder to read.
- Do not overuse it: when the type is important but unclear, an explicit declaration may communicate intent better.
- A function's return type or parameter type cannot be left ambiguous in the ordinary examples shown in this lecture; `auto` is not a universal replacement for type declarations.

6. Structured bindings

Structured bindings let you introduce named variables from the elements of a pair or a suitable struct-like object.

```
auto p = std::make_pair(std::string("s"), 5);
auto [text, number] = p;

// text is a string; number is an int
```

Using structured bindings with the quadratic example

```
auto [found, solutions] = quadratic(a, b, c);

if (found) {
    auto [x1, x2] = solutions;
    std::cout << x1 << " " << x2 << std::endl;
} else {
    std::cout << "No solutions found!" << std::endl;
}
```

The benefit is semantic clarity: names such as `found`, `x1`, and `x2` are easier to understand than repeated `first` and `second` accesses.

7. Common pitfalls and review

- **Integer division:** `5 / 2` is 2, not `2.5`.
- **Overloading:** the parameter list distinguishes overloads; changing only the return type is insufficient.
- **Struct initialization:** brace-initializer values follow the declared field order.
- **auto:** the compiler still assigns a definite type.
- **Structured bindings:** use meaningful names to improve readability.

Quick self-check

- What is the value and type of `5 / 2`? What about `5.0 / 2`?
- Why is a `Student` struct better than passing name, state, and age as unrelated parameters everywhere?
- When would `auto` improve readability, and when might it reduce clarity?